

Placement Portal

Modern Application Development I - Project Report

A Flask-based campus recruitment management system

Author

Full Name	Archita Jain
Roll Number	23f3001889
Email	23f3001889@ds.study.iitm.ac.in

AI / LLM Usage

Claude (Anthropic) was used as a development assistant, primarily for debugging Jinja2 template logic, resolving Flask-RESTful request parser issues, and generating repetitive HTML/Bootstrap boilerplate for the admin CRUD views. All design, logic, debugging, and integration were done manually. Estimated usage: ~20%, limited to code suggestions and documentation formatting.

Project Description

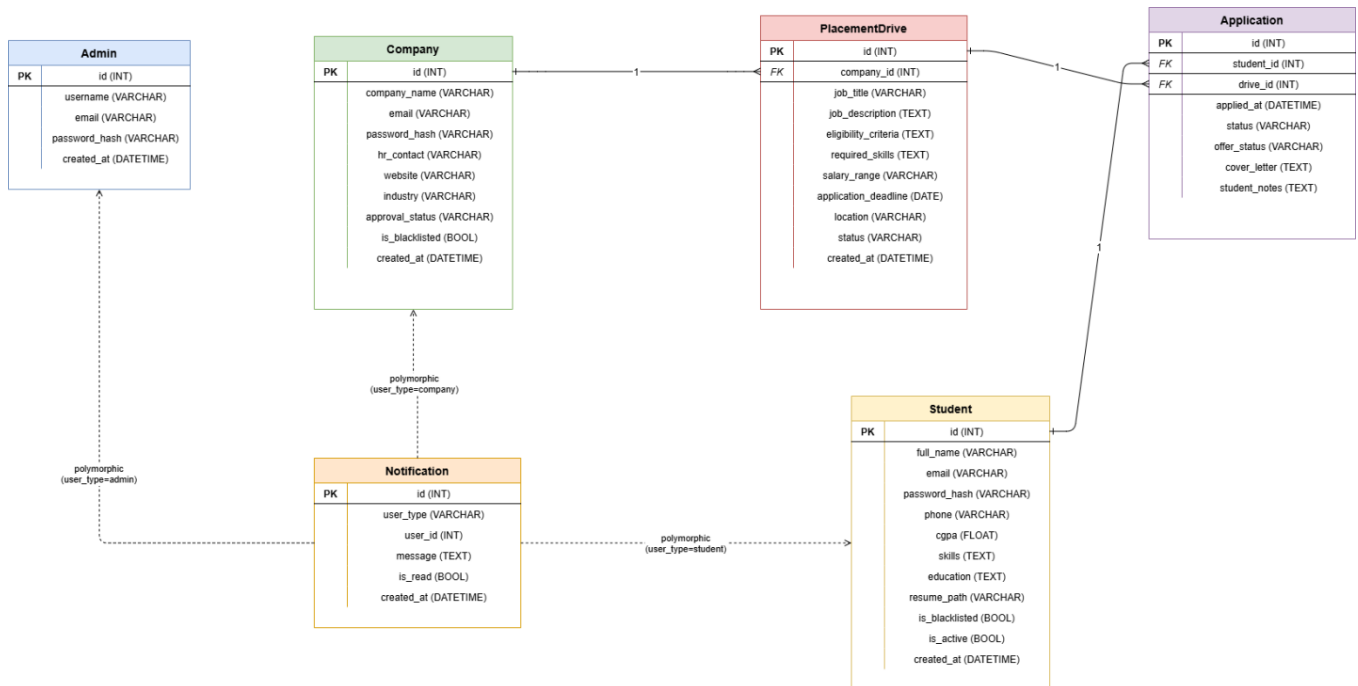
The Placement Portal is a multi-role web application for managing campus recruitment. It supports three user types — Admin, Company, and Student — and provides a centralised platform for posting placement drives, managing applications, and tracking candidate progress through the hiring pipeline.

Technologies Used

Technology	Purpose / Reason
Python / Flask	Core web framework — lightweight, easy routing via Blueprints
Jinja2	Server-side HTML templating integrated with Flask
SQLite + SQLAlchemy	Relational database with ORM; SQLite chosen for zero-config local development
Flask-Login	Session management and role-based access control
Flask-RESTful	REST API layer for programmatic access to students and drives
Werkzeug	Password hashing (PBKDF2) and secure file upload utilities
Bootstrap 5	Responsive UI components; reduces custom CSS effort
Bootstrap Icons	Icon set consistent with Bootstrap 5 design language

DB Schema Design

The schema consists of five models. The DB schema diagram is attached separately.



Design rationale: Company and Student are separate tables (not a polymorphic users table) to keep role-specific fields clean. Applications enforce a unique constraint on (student_id, drive_id) to prevent duplicate submissions. Notifications use a generic (user_type, user_id) pair with a composite index, avoiding a full join when fetching per-user alerts. Cascade deletes on Drive→Application and Company→Drive ensure referential integrity without orphan rows.

API Design

A REST API is exposed at /api/ using Flask-RESTful. The following resources are implemented:

Endpoint	Methods	Description
/api/stats	GET	Returns aggregate counts — students, companies, drives, applications
/api/students	GET, POST	List all students; create a new student with full_name, email, password, cgpa, skills
/api/students/<id>	GET, PUT, DELETE	Retrieve, update, or delete a specific student by ID
/api/drives	GET, POST	List all drives; create a new drive with job_title and company_id
/api/drives/<id>	GET, PUT, DELETE	Retrieve drive; update its status (Approved/Rejected/Closed); delete drive

All endpoints accept and return JSON. Request parsing uses Flask-RESTful's RequestParser. The YAML (OpenAPI) definition is submitted as a separate file alongside this report.

Architecture and Features

Project Structure

The application follows Flask's Application Factory pattern. app.py defines create_app(), which initialises extensions, registers Blueprints, and sets up error handlers. Business logic is split into four Blueprints under the routes/ directory: auth (login, register, logout), admin (dashboard, student/company/drive management), company (drive CRUD, applicant management), and student (browse drives, apply, profile, notifications). Jinja2 templates mirror this structure under templates/, with a shared base.html providing the navbar, flash messages, Bootstrap CDN, and page blocks. Static files (uploaded resumes) are stored under static/uploads/resumes/.

Features Implemented

Authentication & Authorisation

- Role-based login (Admin / Company / Student) via a single login form with radio selection
- Password hashing with Werkzeug PBKDF2; minimum 6-character enforcement on registration
- Flask-Login session management; per-Blueprint decorator guards (admin_required, company_required, student_required)
- Company accounts require admin approval before login is permitted; blacklisted accounts are blocked at login

Admin Panel

- Dashboard with real-time stats (total students, companies, drives, applications)
- Approve / Reject individual or bulk companies and drives
- Search and filter students and companies; blacklist / unblacklist any account
- View full student profile and application history; download/preview resume
- Embedded API request builder on the dashboard for live endpoint testing

Company Panel

- Post new placement drives (pending admin approval); edit, close, re-open, or delete drives
- View applicants per drive with tab-based status filters and CGPA sorting
- Update individual or bulk applicant status; automated notifications sent on every change
- View student profiles and resumes restricted to applicants of own drives

Student Panel

- Browse and search approved drives; apply with an optional cover letter (one application per drive enforced)
- Application history with status badges, offer acceptance/decline workflow, and personal notes
- Notification centre — unread notifications surfaced on dashboard, marked read on notification page
- Profile management with resume upload (PDF/DOC/DOCX); resume viewable inline

Video

<https://drive.google.com/file/d/1g80tOs4j0xVdT5De7PpWF9nVWSRo4q3C/view?usp=sharing>